

Arbres Lexicogràfics Eficients: Informe

GRAU A – Q1 CURS 2025-2026

Departament de Ciències de la Computació
Universitat Politècnica de Catalunya

NIL AVIÑÓ

nil.avino@estudiantat.upc.edu

ARNAU CULLELL

arnau.cullell@estudiantat.upc.edu

LAIA GINABREDA

laia.ginabreda@estudiantat.upc.edu

ANDREU MORAN

andreu.moran@estudiantat.upc.edu

24/10/2025

Resum

Aquest treball és un estudi profund i formal sobre els tries, estructures de dades que permeten emmagatzemar i cercar trossos de text (paraules o frases) de forma eficient. Els objectius principals han estat entendre els tries i les seves variants, comparar-los amb estructures similars i analitzar-ne l'eficiència teòrica i empírica, justificant de forma crítica les decisions preses. La metodologia per desenvolupar el treball es pot dividir en tres fases: cerca intensiva d'informació per a poder entendre conceptes i aclarir aquells confusos; selecció de dos tries a desenvolupar, fer el seu codi i després implementar millores; anàlisi i comparació dels resultats obtinguts després de fer proves amb uns textos específics. Els resultats obtinguts al final ens permeten afirmar que en general el Radix trie és més ràpid i ocupa menys espai que el trie clàssic.

Part I. Informe Tècnic

1 Introducció i Objectius

En aquest projecte el principal problema a resoldre és el paradigma dels tries, on no hi ha un trie perfecte pel que fa a l'eficiència. Si prioritzem millorar el temps de cerca llavors perdrem una quantitat d'espai de disc considerable, i a l'inrevés succeeix el mateix. Si es vol un trie que tingui els dos conceptes balancejats, cal implementar per exemple un Patricia trie, el qual, encara que sigui molt interessant, és molt complicat d'implementar en el poc temps donat per fer el projecte.

Respecte a l'assignatura d'Algorísmia, realitzada en la Facultat d'Informàtica de Barcelona, el tema dels tries no és explicat a teoria ni en cap classe de problemes, sinó que és un tema on l'autoaprenentatge juga un paper fonamental. Cal dir que gràcies a altres conceptes vists a l'assignatura, i a coneixements previs d'Estructures de Dades i Algorismes, ha sigut més fàcil entendre i assolir el concepte de trie i de com funciona internament.

Els tries són estructures de dades àmpliament utilitzades en tasques com autocompletar paraules o corregir-les si tenen un error ortogràfic; trobar seqüències d'ADN importants que poden determinar si una persona serà propensa a tenir una malaltia o no; estan implementades en els sofisticats cercadors de Google i permeten fer cerques més ràpides; serveixen per poder comprimir

fitxers fent un canvi de llenguatge a binari per exemple; també són útils en l'àmbit de xarxes de computadors, on es poden usar per a l'encaminament d'adreces IP.

L'objectiu principal d'aquest projecte és el d'entendre com funcionen internament els tries, avaluar-ne l'eficiència i veure les aplicacions pràctiques que tenen en el món real. Altres objectius interessants són: conèixer les diferents variants dels tries i les seves característiques; decidir el rumb del projecte amb decisions raonades; saber analitzar formalment els costos asimptòtics de cada trie.

2 Antecedents

Per a entendre el projecte cal remarcar primer uns quants conceptes previs. Primer de tot, un trie (prové de *Retrieval* i es pronuncia com *try*) és un tipus d'estructura de dades arbòria la qual permet la recuperació d'informació (trossos de text) de forma eficient. El funcionament d'un trie naïf o clàssic és molt senzill. Cada node té un valor, que normalment indica en quina línia del text surt la paraula, i un vector de mida R, on normalment R és 256 i serveix per representar el codi ASCII estès. Llavors, cada posició del vector té com a clau una lletra ASCII, i la informació de cada element del vector conté un punter fins al següent node.

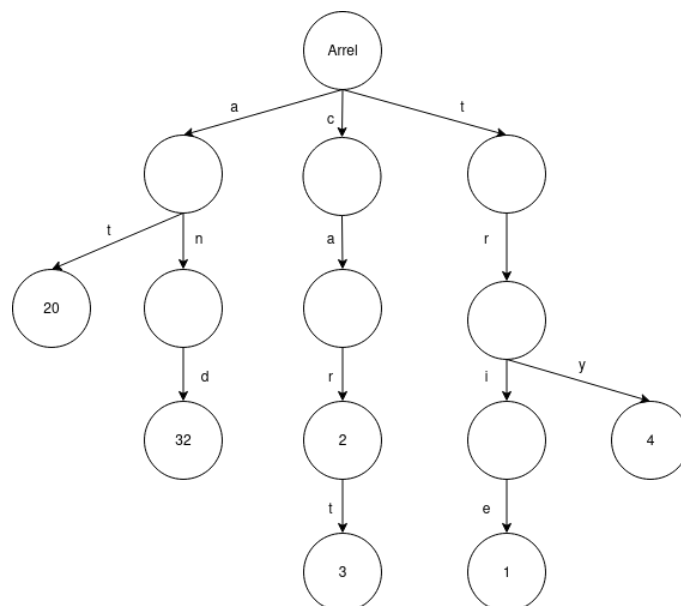


Figura 1: Exemple de trie clàssic

A continuació tenim els principals tipus de tries que hi ha:

- **Trie clàssic:** és el trie més simple de programar i entendre. Com s'ha mencionat amb anterioritat, cada node té un vector de mida R, la qual cosa implica que cada node tindrà R fills, els quals la majoria normalment seran NULL. És molt ràpid buscant valors, però, també comporta una gran pèrdua de memòria, i per tal de fer que ocupi menys es pot usar un map o una llista enllaçada, però sacrificuem eficiència de cerca. En la pràctica se solen usar per exemple en l'autocompletat de cercadors o d'editors de codi.
- **Ternary Search Trie:** aquest és una mescla entre un *binary search tree* i un trie clàssic. Cada node només tindrà un valor, un caràcter, i tres nodes fills: *smaller* (esquerra), *equal* (mig) i *larger* (dreta). Se'n va d'un node a un altre segons el caràcter que guarda el node pare i es compara amb el dels fills. Ocupen bastant menys espai que els tries clàssics i tampoc

perden molta eficiència en la cerca. Tenen un ús molt similar al trie clàssic, però s'usen més en dispositius que tinguin poca memòria o que siguin més senzills.

- **Radix Trie:** variant del trie clàssic on cada node amb només un fill es fusiona amb el seu pare. Un altre canvi respecte al normal és que cada fill com a molt tindrà r fills (radix), on $r = 2^x$, per a tota $x > 0$. Si la $r = 2$ es diu que el radix trie és binari, si no, si $r \geq 4$ el radix trie llavors és r -ary. Quan r és gran el trie resultant és més dispers i menys profund, i quan r és petita és menys dispers i més profund, l'ideal seria trobar un equilibri entre els dos. Els radix trie són molt més eficients que els clàssics, ja que permet fer salts de nodes innecessaris que només tenen un fill. Normalment, se solen usar en routers o xarxes per tal d'emmagatzemar les IP per prefixos, o també en servidors DNS per trobar dominis ràpidament.
- **Patricia Trie:** tipus de radix trie especial, on radix = 2, per tant, és un trie binari. Ara el que es guarda dins del node és un nombre (*skip number*) que indica la posició del primer caràcter que canvia i diferencia als seus fills. Per passar d'un node a un altre es fa indicant un caràcter, i com és un trie binari, només serveix amb alfabets amb dos símbols, com per exemple els alfabets $\{a,b\}$ o $\{0,1\}$. Tenen l'avantatge que per cada paraula nova només es crea un node, la qual cosa fa que sigui molt més compacte. Es poden usar per a la compressió d'informació o en sistemes d'arxius del kernel.

Sigui N = nombre de paraules en el text, L = longitud d'una paraula, i R = nombre de símbols al text, llavors, aquests són els costos asimptòtics que tenen per a quan troben la paraula, quan no la troben, quan insereixen una paraula i el cost en memòria:

	Troba paraula	No troba la paraula	Insertar	Memòria
Trie Clàssic	$O(L)$	$O(\log_R(N))$	$O(L)$	$O(N*(R + 1))$
Ternary Search Trie	$O(L + \ln(N))$	$O(\ln(N))$	$O(L + \ln(N))$	$O(4N)$
Radix Trie (compressed)	$O(L)$	$O(L)$	$O(L)$	$O(N*L)$
Patricia Trie (radix = 2)	$O(L*\log_2(R))$	$O(L*\log_2(R))$	$O(L*\log_2(R))$	$O(N)$

Taula 1: Costos asimptòtics

Observació pel Patricia Trie: si l'alfabet té mida R , cada caràcter es pot codificar amb $\log_2(R)$ bits. Per a una paraula de longitud L , el nombre total de bits és $= L * \log_2(R)$, per tant, es pot dir que el cost de trobar una paraula i d'inserir és el nombre de bits.

Les principals fonts per a trobar aquesta informació han sigut: la presentació *Algorithms* de Robert Sedgewick i de Kevin Wayne; la wikipedia per trobar informació general dels tries; geeksforgeeks per veure exemples; la presentació de Yufei Tao sobre tries i patricia tries, de la Universitat de Queensland.

3 Disseny i Implementació

Decisió de la segona implementació

Com a línia base s'ha implementat un R -way trie (trie clàssic) per la seva simplicitat, cost temporal previsible i interfície mínima (recorregut símbol-a-símbol amb `next[c]` indexat per byte). Com a segona estructura s'ha triat un radix trie, que manté la semàntica de prefix però aplica compressió de camins (etiquetes d'arestes com a cadenes) i divisió d'arestes en coincidència parcial, reduint nodes intermedis i millorant la localitat de memòria. Aquesta elecció és més adequada per conjunts amb prefixos compartits i alfabets no trivials. El Patricia trie (base 2) es descarta per desalineació amb l'alfabet i sobrecost d'implementació sense guany proporcional en el nostre escenari.

Estructures i representació interna

·R-way trie

-Node:

- **valor:** vector d'enters que conté les línies on surt la paraula o prefix que representa el node.
- **next:** vector d'enllaços de mida R indexats per `unsigned char`.
- **es paraula:** Boleà que determina si el node es paraula o prefix en el text.

-Rutina de descens:

- A cada nivell d es consulta `key[d]` i es baixa per `next[key[d]]`.

·Radix trie

-Node:

- **valor:** vector d'enters que conté les línies on surt la etiqueta (i per tant els prefixos de l'etiqueta).
- **label:** Un string que representa la seqüència de símbols consecutius al node anterior.
- **child:** diccionari *caràcter inicial* $\rightarrow$ *fill* corresponent.
- **es paraula:** Boleà que determina si el node es paraula o prefix en el text.

-Rutines clau:

- **cpl(a,b):** longitud del prefix comú entre la cadena pendent i l'etiqueta;
- **Edge split:** creació d'un node intermedi si la coincidència és parcial.

Funcions implementades

Inserció:

- **R-way trie:** creació de nodes al vol i avanç símbol-a-símbol ($d \rightarrow d+1$); al node final s'afegeix l'identificador. **Cost:** $O(L)$; constant baixa si R és petit, amb risc d'espai desaprofitat en escenaris esparsos.
- **Radix trie:** s'afegeix el valor al node actual (propagant consultes de prefix). Casos: (i) no hi ha aresta pel caràcter inicial $\Rightarrow$ es crea una aresta amb el sufix; (ii) l'etiqueta es consumeix completament $\Rightarrow$ descens al fill; (iii) coincidència parcial $0 < \ell < |etq| \Rightarrow$ *split* i reetiquetatge. **Cost:** $O(L)$ en caràcters; comparació per blocs amb menys nodes visitats en la pràctica.

Cerca exacta:

- **R-way trie:** recorregut símbol-a-símbol fins $d = |key|$; si el node existeix, el seu **valor** conté el número de cada línia on surt la paraula cercada.
- **Radix trie:** consum d'etiquetes comprimides; qualsevol desajust o aturada a mig etiqueta implica absència de coincidència; si s'arriba al node representant la clau, es retorna **valor**.

Autocompletat:

- **R-way trie:** al no trobar la paraula exacte la funció d'autocompletar retorna una llista de paraules que comencen amb el la paraula no trobada. S'aplica un DFS fins trobar un maxim de paraules (nosaltres posem el màxim a 5) per tal de no retornar una llista molt gran en cas de que la paraula no trobada sigui un prefix de moltes altres paraules.
- **Radix trie:** funciona de la mateixa forma que el R-way trie. Al posar un tros d'una paraula sense acabar et retorna una llista de 5 suggerències sobre quina paraula podries estar buscant. Cost esperat $O(|p| + k)$ (prefix $|p|$ i k resultats).
- En ambdós casos es va considerar implementar un autocompletat basat en freqüència, però es va descartar per no aportar un guany significatiu.

Garantia d'eficiència: invariants, decisions i costos

R-way trie. **Invariants:** (i) cada nivell consumeix un caràcter; (ii) `next` indexa directament per `unsigned char`. **Implementació:** inserció recursiva amb creació al vol i camí determinista `next[key[d]]`. **Costos:** inserció/cerca $O(L)$; espai $O(N \cdot R)$ en el pitjor cas espars (molts `nullptr`).

Radix trie. **Invariants:** (i) etiquetes no buides; (ii) *splits* que preserven ordre i valors; (iii) `cpl` estableix el tall de manera unívoca. **Implementació:** `cpl` guia la inserció i els *splits*; la cerca consumeix etiquetes completes. **Costos:** temps $O(L)$ amb comparació per blocs; espai inferior al trie clàssic quan hi ha prefixos llargs compartits; millor localitat de memòria.

Comparació qualitativa i aplicabilitat

	R-way trie	Radix trie
Escenaris forts	Alfabet petit i dades denses; latència baixa per accés directe <code>next[c]</code> .	Prefixos molt compartits; alfabets grans o esparsos; memòria limitada; millor localitat.
Escenaris dèbils	Alfabet gran amb esparsitat (molts enllaços buits).	Claus molt curtes sense prefixos comuns (benefici de compressió marginal).
Cost temporal	$O(L)$ símbol-a-símbol (constant baixa).	$O(L)$ amb comparació per blocs; menys nodes visitats en pràctica.
Cost espacial	Pot créixer amb R i l'esparsitat.	Més compacte en presència de camins comuns.
Implementació	Molt directa; codi curt i mantenible.	Més complex (<code>cpl</code> i <i>split</i>); flexible i escalable.

Alternatives

Patricia (trie binari): òptim quan les claus es processen bit a bit; desaconsellat en alfabet caràcter general. Implementació massa complicada pels escenaris que volem comprovar.

Disciplina d'implementació i correcció

R-way: la inserció crea nodes al vol i evita duplicats al node final; el descens determinista garanteix correcció en la cerca. *Radix*: `cpl` fixa el punt de divisió; el *mid-node* hereta valors per mantenir la propietat de prefix; la cerca només admet coincidències exactes d'etiqueta.

4 Avaluació Experimental

Objectius i hipòtesis

Avaluem empíricament el rendiment en **consulta** de les dues estructures (*R-way trie* i *radix trie*) en termes de: (i) latència per consulta, (ii) nodes visitats, (iii) profunditat mitjana recorreguda, (iv) ús de memòria.

Hipòtesis: el *radix trie* visitarà menys nodes i tindrà latències inferiors o comparables en conjunts amb prefixos compartits i/o alfabets grans; el *R-way* pot presentar constants més baixes en conjunts densos i alfabet petit.

Conjunts de dades

Considerem conjunts parametritzats per N (entrades), L (longitud mitjana) i alfabet $|\Sigma|$:

- **Baix grau de prefixos compartits** (strings quasi uniformes).
- **Alt grau de prefixos compartits** (p. ex., *urls*, rutes, claus jeràrquiques).
- **Alfabet petit** (bases nitrogenades: A, C, G, T) i **gran** (ASCII estès).

Protocol de proves de consulta

Per a cada conjunt:

1. **Cerca exacta:** mostreig de 150 claus existents i 150 inexistents; mesura de latència mitjana i ocurrencies, nodes visitats per operació i profunditat mitjana del camí recorregut.
2. **Consultes de prefix:** mostreig de 300 prefixos (de longituds variades); latència fins a localitzar el node de prefix (sense enumerar resultats) i nodes visitats.
3. **Autocompletat 5 paraules:** temps fins retornar 5 elements i nodes visitats addicionals a partir del node de prefix.

Mètriques utilitzades:

-**Latència mitjana per operació** (ns/op).

-**Nodes visitats de mitjana per operació i profunditat mitjana** ($\bar{d}$) en el recorregut.

-**Memòria total** (MB) i bytes per entrada. En el nostre cas 5MB per entrada ja que el R-Way Trie no ho podia processar sinó. Cal remarcar que el Radix podia llegir fins i tot el document de la Wikipedia donat a l'enunciat del projecte.

Resultats amb baix grau de prefixos compartits: ($N=300$, $L=100$, $|\Sigma|=62$)

- **Cerca exacta:**

Estructura	Temps mitjà/op (ns)	Nodes visitats de mitjana/op
R-way	6544.99	54.33
Radix	1503.83	4.97

Taula 2: Latència mitjana (ns) i nodes visitats de mitjana per operació.

- **Consultes de prefix i autocompletat:**

Estructura	Prefix-hit (ns)	5 sugg. (ns)	Prefix nodes/op	Sugg. nodes/op
R-way	432.04	37002.08	10.13	116.27
Radix	361.40	447.41	4.86	0.86

Taula 3: Temps fins al node de prefix, fins a 5 suggeriments, nodes de mitjana fins arribar al prefix, nodes de mitjana visitats per l'autocompletat.

- **Profunditat i memòria**

Estructura	$\bar{d}$	$d-max$	#nodes	MB
R-way	54.33	98	4212795	539.23
Radix	4.97	7	69137	11.61

Taula 4: Profunditat mitjana $\bar{d}$, profunditat màxima $d-max$, #nodes totals i memòria (MB).

Resultats amb alt grau de prefixos compartits: ($N=300$, $L=30$, $|\Sigma|=39$)

- Cerca exacta:

Estructura	Temps mitjà/op (ns)	Nodes visitats de mitjana/op
R-way	1124.13	15.65
Radix	1061.58	6.418

Taula 5: Latència mitjana (ns) i nodes visitats de mitjana per operació.

- Consultes de prefix i autocompletat:

Estructura	Prefix-hit (ns)	5 sugg. (ns)	Prefix nodes/op	Sugg. nodes/op
R-way	334.34	8403.88	11.54	23.56
Radix	376.86	601.33	5.94	1.75

Taula 6: Temps fins al node de prefix, fins a 5 suggeriments, nodes de mitjana fins arribar al prefix, nodes de mitjana visitats per l'autocompletat.

- Profunditat i memòria

Estructura	$\bar{d}$	$d-max$	#nodes	MB
R-way	15.65	27	2630873	336.75
Radix	6.41	9	254525	42.76

Taula 7: Profunditat mitjana $\bar{d}$, profunditat màxima $d-max$, #nodes totals i memòria (MB).

Resultats amb alfabet petit: ($N=300$, $L=250$, $|\Sigma|=4$)

- Cerca exacta:

Estructura	Temps mitjà/op (ns)	Nodes visitats de mitjana/op
R-way	25894.69	152.62
Radix	2538.63	8.85

Taula 8: Latència mitjana (ns) i nodes visitats de mitjana per operació.

- Consultes de prefix i autocompletat:

Estructura	Prefix-hit (ns)	5 sugg. (ns)	Prefix nodes/op	Sugg. nodes/op
R-way	977.89	193905.54	12.08	500.91
Radix	380.77	675.49	7.90	2.51

Taula 9: Temps fins al node de prefix, fins a 5 suggeriments, nodes de mitjana fins arribar al prefix, nodes de mitjana visitats per l'autocompletat.

- **Profunditat i memòria**

Estructura	$\bar{d}$	$d-max$	#nodes	MB
R-way	152.62	251	5089017	651.39
Radix	8.85	12	33873	5.69

Taula 10: Profunditat mitjana $\bar{d}$, profunditat màxima $d-max$, #nodes totals i memòria (MB).

Resultats amb alfabet gran: ($N=300$, $L=15$, $|\Sigma|=256$)

- **Cerca exacta:**

Estructura	Temps mitjà/op (ns)	Nodes visitats de mitjana/op
R-way	642.41	8.36
Radix	640.49	6.67

Taula 11: Latència mitjana (ns) i nodes visitats de mitjana per operació.

- **Consultes de prefix i autocompletat:**

Estructura	Prefix-hit (ns)	5 sugg. (ns)	Prefix nodes/op	Sugg. nodes/op
R-way	1265.60	1864.90	9.58	4.42
Radix	1593.16	420.32	7.87	1.21

Taula 12: Temps fins al node de prefix, fins a 5 suggeriments, nodes de mitjana fins arribar al prefix, nodes de mitjana visitats per l'autocompletat.

- **Profunditat i memòria**

Estructura	$\bar{d}$	$d-max$	#nodes	MB
R-way	8.36	32	1027810	131.55
Radix	6.88	17	478058	80.31

Taula 13: Profunditat mitjana $\bar{d}$, profunditat màxima $d-max$, #nodes totals i memòria (MB).

Discussió i interpretació

Amb els resultats obtinguts podem veure que la profunditat mitjana $\bar{d}$ coincideix sempre amb la mitjana de nodes recorreguts per operació. És una cosa que no sembla obvia inicialment, però té sentit perquè quan fem la cerca d'una paraula, cada vegada que visitem un nou node la profunditat augmenta en 1 també i, per tant, al final hem visitat tants nodes com nivells hem baixat al trie. A més a més, cal dir que la profunditat i el nombre de nodes visitats sempre coincideix amb la mida de la paraula en el cas del trie clàssic, cosa que encaixa amb el que esperàvem.

També es pot observar com el Radix trie normalment mira molts pocs nodes després de trobar el prefix i fer l'autocompletat. L'explicació d'aquest fenomen resideix en el fet que normalment en els tests de proves els prefixos establerts moltes vegades només retornaven una paraula, i això implica que no calia veure molts més nodes, o directament, no visitar cap node més.

Interpretant les dades es pot dir clarament que el Radix trie és més eficient temporalment i espacialment. El Radix trie pot ser 10 vegades més ràpid i ocupar 115 vegades menys memòria

que el trie clàssic. Cal mencionar que no hi ha molta diferència de temps i espai en els casos de l'alfabet gran i l'alt grau de prefixos compartits. Aquesta semblança de resultats pot ser donada per la mida de les paraules als textos, ja que com no són gaire llargues no poden permetre veure la diferència entre un trie i l'altre. A continuació uns gràfics per poder observar de millor forma els resultats a cada escenari:

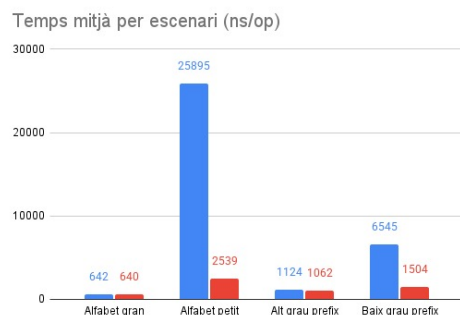


Figura 2: Temps mitjà

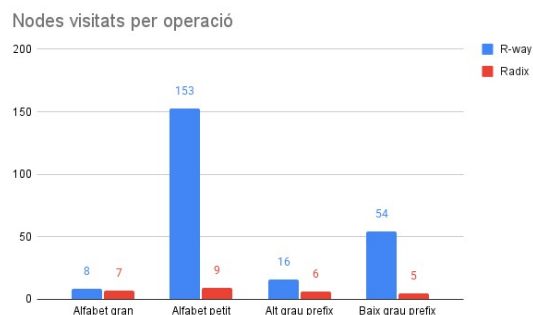


Figura 3: Nodes visitats de mitjana

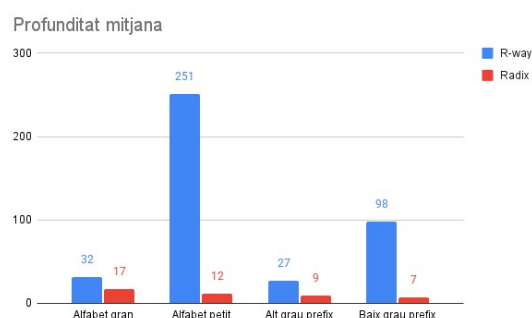


Figura 4: Profunditat mitjana

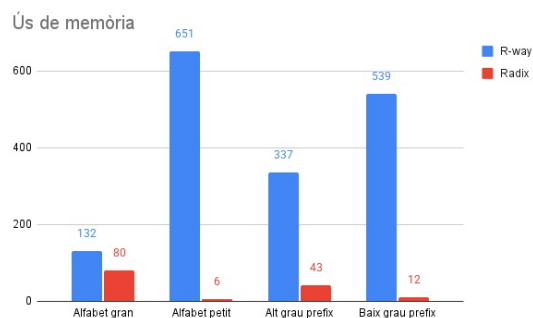


Figura 5: Ús de memòria

La primera hipòtesi plantejada afirmava que el Radix trie visitaria menys nodes i tindria menys latència que el trie clàssic en els casos de l'alfabet gran i amb molts prefixos compartits. Finalment podem afirmar que la hipòtesi es compleix, visita molts menys nodes i els temps de cerca són millors, encara que siguin molt semblants.

La segona hipòtesi deia que el trie clàssic seria millor en el cas de l'alfabet petit i en conjunts densos. Tampoc es compleix, ni en aquest dos casos ni en els altres dos, el radix trie ha demostrat ser molt millor del que esperàvem.

Sobre consultes de frases i de sufixos: viabilitat i cost de memòria

Frases (multi-paraula). Un trie per tokens pot indexar frases com a claus completes; però indexar *totes* les frases internes (tots els n -grams) escala com $O(\sum_i |s_i|)$ en memòria per nivell d' n , amb constants elevades i duplicació de prefixos. Alternatives:

- **Trie només de claus canòniques** (frases completes): memòria acotada; però les cerques d' n -grams interns requereixen preprocés o postprocés.
- **Índex d' n -grams acotat** (p.ex. $n \leq 3$): equilibri entre cobertura i memòria.
- **Radix + separadors** (p.ex. - entre tokens): manté compressió; segueix existint el cost d'explosió si s'indexen tots els n -grams.

Sufixos i subcadenaes. Les estructures trie clàssiques estan dissenyades per **prefixos**. Per **sufixos/substrings**, les opcions naturals són:

- **Trie de sufixos / arbre de sufixos:** cerca de substring en temps lineal en la longitud de la consulta, però amb un cost de memòria elevat (constant gran sobre $O(n)$).
- **Índex invers d' n -grams** (per subcadenaes): similar al cas de frases; la memòria creix amb els grams.
- **Trie de claus invertides** (reversar claus): eficient per *sufix* pur, però no resol *qualssevol* substrings.

Conclusió: donar suport general a frases internes i substrings implica un **creixement significatiu de memòria** i duplicació d'informació. Per aquest projecte prioritzem **prefixos i cerca exacta**; extensions a frases/sufixos s'han de fer amb índexs addicionals i/o n -grams acotats segons les restriccions de memòria.

Amenaces a la validesa

- **Interna:** soroll de mesura (timers, *cache*).
- **Externa:** generalització a altres conjunts/alfabets.
- **Constructe:** definició de memòria (RSS vs. comptadors d'*allocator*). (Mitigació: rèpliques, publicació de conjunts i scripts.)

5 Conclusions

Després de tot el projecte, podem afirmar que el Radix trie serà en general més ràpid, ocuparà menys espai, visitarà menys nodes i tindrà una profunditat menor que el trie clàssic. A més a més, ha quedat demostrat que el trie clàssic també pot ser molt eficient en la cerca de paraules, i fer-ho en $O(L)$, com s'havia vist anteriorment en la taula de costos.

Durant el projecte, el grup ha assolit un gran grau de rendiment, on tothom ha aportat i cadascú ha fet i assolit la seva part. Alguns han fet més codi, altres han fet més text, però no es pot dubtar que hem treballat en equip i ens hem comunicat molt bé durant el treball sencer. La major dificultat o obstacle ha sigut el temps i el context, ja que volíem fer un projecte més ambiciós implementant un Patricia trie, però estant a la universitat i amb altres projectes se'ns ha fet impossible arribar a aquell nivell desitjat. També, polir detalls o fer més implementacions era una idea que teníem, però com acabem d'explicar, el temps i el context no ens han acompanyat.

Algunes millores interessants que es podrien haver fet al treball són: canviar o modificar la R del R-Way trie (el clàssic) per cada prova, ja que cadascuna té un alfabet de mida diferent, i potser creava més memòria de la necessària; fer més tests de proves amb diferents mides, ja que nosaltres només hem fet els tests de proves amb fitxers de 5MB per tal d'assegurar que el trie clàssic ho pogués soportar, però finalment considerem que aquesta seria una expansió interessant pel projecte en un futur; posar paraules més llargues als tests de proves, perquè hem pogut observar que als tests on les paraules tenien una major longitud el radix trie li treia molta avantatge al trie clàssic, i creiem que fer aquest canvi permetria una millor comparació entre els tries.

Finalment, aquesta anàlisi evidencia que l'elecció entre un trie i un altre depèn del context i on es vulguin utilitzar, però és innegable que el Radix serà sempre o gairebé sempre una millor opció, ja que ofereix un millor compromís entre temps i espai.

Part II. Informe d'Autoaprenentatge

6 Descripció i valoració del procés d'autoaprenentatge

6.1 Metodologia

El nostre equip va organitzar el projecte de manera col·laborativa i iterativa. En una primera fase uns es van encarregar de la cerca bibliogràfica i comprensió teòrica dels tries, mentre que altres van començar amb la planificació de la implementació del *R-way trie*.

Per coordinar-nos vam utilitzar **GitHub** per gestionar el codi i fer seguiment dels canvis, i **Google Docs** per compartir notes i possibles idees abans de implementar-les. Les reunions de posada en comú es van fer de manera periòdica per revisar els progressos, definir nous objectius i corregir possibles errors de disseny.

El desenvolupament es va plantejar de forma **incremental**: primer vam implementar un trie bàsic i funcional, i després vam decidir la segona implementació, el *radix trie*, per últim vam pensar que seria interessant fer l'autocompletat. Aquesta estratègia va permetre validar cada etapa abans de continuar i reduir el risc d'errors acumulats.

Per a l'experimentació, vam definir conjunts de dades amb diferents característiques (alfabets petits i grans, paraules amb prefixos comuns o no) i vam mesurar el rendiment amb temporitzadors interns del llenguatge. Finalment, vam analitzar els resultats en taules i gràfics, contrastant-los amb les cotes asimptòtiques obtingudes a la fase teòrica.

6.2 Valoració del procés d'autoaprenentatge

Aquest projecte ha estat una experiència molt enriquidora, tant a nivell tècnic com personal. Hem après a **interpretar, dissenyar i comparar estructures de dades avançades** que no havíem treballat a classe, com els *radix tries* o els *Patricia tries*, aplicant coneixements previs d'algorismes i estructures de dades.

El repte principal va ser entendre com optimitzar espai i temps en un context real, i com representar estructures amb una alta eficiència de memòria. La lectura de fonts com *Sedgewick & Wayne* o *GeeksforGeeks* ens va ajudar a comprendre millor els detalls teòrics i a traduir-los en decisions pràctiques de codi.

També hem millorat habilitats **transversals** com la gestió del temps, la comunicació dins l'equip i l'ús d'eines col·laboratives. El treball en grup ens ha obligat a consensuar decisions tècniques, a ser crítics amb les propostes dels companys i a justificar cada elecció.

Finalment, considerem que hem assolit plenament els objectius inicials: hem entès el funcionament intern dels tries, n'hem implementat dues versions funcionals i hem pogut avaluar-les empíricament. El procés d'autoaprenentatge ha estat constant, i la satisfacció d'obtenir resultats coherents amb la teoria ha estat una de les parts més motivadores del projecte.

Bibliografia

Autor/s principal/s (si és explícit). Títol del web [en línia]. Edició o versió. Lloc , data de publicació. Data d'actualització/revisió [Data de consulta]. Disponible a: adreça URL.

- Sedgewick, Robert, and Kevin Wayne. Algorithms, Addison-Wesley [en línia]. Section 5.3: Tries. [Consultat 26 setembre 2025]. Disponible a: <https://algs4.cs.princeton.edu/lectures/keynote/52Tries.pdf>
- Javier Campos. 2.4 Arboles digitales de busqueda, tries y patricia [en línia]. Universidad de Zaragoza. [Consultat 5 octubre 2025]. Disponible a: <https://webdiis.unizar.es/asignaturas/TAP/material/2.4.digitales.pdf>
- Trie Data Structure - GeeksForGeeks [en línia]. Última modificació 1 setembre 2025. [Consultat 5 octubre 2025]. Disponible a: <https://www.geeksforgeeks.org/dsa/trie-insert-and-search/>
- Trie - Wikipedia [en línia]. [Consultat 7 octubre 2025]. Disponible a: <https://en.wikipedia.org/wiki/Trie>
- Radix tree - Wikipedia [en línia]. [Consultat 7 octubre 2025]. Disponible a: https://en.wikipedia.org/wiki/Radix_tree
- Ali Masri. Exploring the Power Tries [en línia]. Medium. [Consultat 9 octubre 2025]. Disponible a: <https://medium.com/towards-data-engineering/exploring-the-power-of-tries-493adc52022e>
- Yufei Tao. Tries, Patricia Tries, and Suffix Trees [en línia]. University of Queensland. [Consultat 10 octubre 2025]. Disponible a: <https://www.cse.cuhk.edu.hk/~taoyf/course/comp3506/camp/trie.pdf>

Apèndixs

Deixem alguns gràfics i resultats interessants que poden resumir millor els experiments fets:

Escenari	Trie	Temps (ns/op)	Nodes visitats	Profunditat	Memòria (MB)
Alfabet gran	R-way	642,41	8,36	32	131,55
Alfabet gran	Radix	640,49	6,67	17	80,31
Alfabet petit	R-way	25894,69	152,62	251	651,39
Alfabet petit	Radix	2538,63	8,85	12	5,69
Alt grau prefix	R-way	1124,13	15,65	27	336,75
Alt grau prefix	Radix	1061,58	6,41	9	42,76
Baix grau prefix	R-way	6544,99	54,33	98	539,23
Baix grau prefix	Radix	1503,83	4,97	7	11,61

Taula 14: Taula resum dels resultats

Eficiència combinada (memòria i temps mitjà)

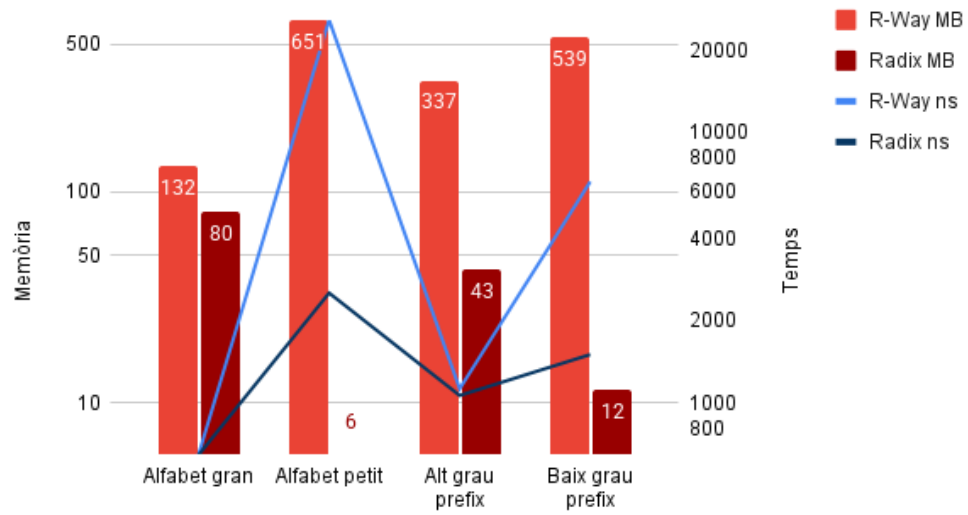


Figura 6: Gràfic resum dels resultats de l'experiment